AutoAIM Studio

User Manual

Automated Machine Learning for Materials Science

César Gabriel Vera de la Garza & Serguei Fomine

Version 1.1.0 | 2026

New Release

Table of Contents

Introduction

AutoAIM Studio is a comprehensive machine learning platform designed for materials science applications. It provides an intuitive graphical interface for data preprocessing, model training, hyperparameter optimization, neural network construction, ensemble modeling, and model interpretation.

Key Features

- Automated Data Preprocessing: Handle missing values, categorical encoding, and feature scaling.
- Custom Composition Feature Engineering: Generate 106 composition-based descriptors from chemical formulas.
- Multiple ML Algorithms: Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost, SVR, Ridge.
- Neural Network Builder: Visual architecture design with customizable layers and hyperparameter optimization.
- Hyperparameter Optimization: Bayesian optimization with Optuna for regressors and neural networks.
- Ensemble Modeling: Combine multiple models with weight optimization for improved predictions.
- Model Interpretability: SHAP values, permutation importance, partial dependence plots.
- Standalone Prediction Mode: Deploy trained models for inference on new data.

System Requirements

> 💡 **Note:** Ensure your system meets these requirements for optimal performance.

- Windows 10/11 (64-bit).
- 8GB RAM minimum (16GB recommended).
- CUDA-compatible GPU (optional, for neural network acceleration).

Getting Started

Launching the Application

1. Double-click the `AutoAIM Studio.exe` file.

2. The main window will open with multiple tabs at the top.

3. Start by loading your data in the Data tab.

Workflow Overview

The typical workflow follows these steps:

- Load Data → Data Tab.
- Preprocess → Data Tab (handle missing values, encode categoricals).
- Generate Features → Data Tab (optional, composition descriptors).
- Train Models → Training Tab or Neural Network Tab.
- Optimize → Optimization Tab (optional, for hyperparameter tuning).
- Create Ensembles → Ensemble Tab (optional, combine models).
- Analyze Results → Results Tab.
- Interpret Models → Explainability Tab.
- Make Predictions → Predict Tab.

> ⚡ **Pro Tip:** Start with a small dataset to familiarize yourself with the workflow before processing large datasets.

Data Tab

The Data Tab is your starting point for loading and preparing your dataset.

Loading Data

1. Click**"Load Data"**button.

2. Select your data file (CSV or Excel format supported).

3. The data preview will appear in the table below.

Supported Formats

- CSV files (.csv).
- Excel files (.xlsx, .xls).

Data Preview

After loading, you'll see:

- Shape: Number of rows and columns.
- Columns: List of all column names.
- Data Types: Automatic detection of numerical and categorical columns.
- Sample Data: First few rows of your dataset.

Target Column Selection

1. Select your target variable from the "Target Column" dropdown.

2. The system will automatically detect if it's a regression or classification problem.

Preprocessing Options

Handle Missing Values

Checkbox: "Handle missing values". When enabled, missing numerical values are imputed with the median. Missing categorical values are imputed with the most frequent value.

Encode Categorical Variables

Checkbox: "Encode categorical variables". When enabled, categorical variables are one-hot encoded. This expands categorical columns into multiple binary columns.

Scale Features

Checkbox: "Scale features". When enabled, features are standardized (zero mean, unit variance). Recommended for algorithms sensitive to feature scales (SVR, Neural Networks).

Generate Composition Features

Checkbox: "Generate composition features (106 descriptors)".**Requires:** A column containing chemical formulas (e.g., "Fe2O3", "SiO2"). When enabled, the system extracts 106 composition-based descriptors including:

- Atomic number statistics (mean, std, min, max).
- Atomic mass statistics.
- Electronegativity statistics.
- Covalent radius statistics.
- Ionization energy statistics.
- And many more material properties.

> **Materials Science Context:** The composition feature generator is a custom implementation designed for materials science applications. It analyzes chemical formulas and computes aggregate statistics based on elemental properties.

Preparing Data

After configuring options:

1. Click "Prepare Data" button.

2. The system will split data into train/test sets (80/20 by default).

3. All selected preprocessing steps are applied.

4. A confirmation message will appear when complete.

Training Tab

The Training Tab allows you to train multiple machine learning models simultaneously.

Model Selection

Select one or more models from the list:

Random Forest: Ensemble of decision trees, good for general-purpose prediction.

Gradient Boosting: Sequential ensemble, often achieves high accuracy.

XGBoost: Optimized gradient boosting, industry standard.

LightGBM: Fast gradient boosting with leaf-wise growth.

CatBoost: Gradient boosting optimized for categorical features.

SVR: Support Vector Regression, good for smaller datasets.

Ridge: Linear regression with L2 regularization, simple baseline.

Training Configuration

- Problem Type: Automatically detected (regression/classification).
- Cross-Validation Folds: Number of CV folds for evaluation (default: 5).
- Random State: Seed for reproducibility (default: 42).

Training Process

1. Select desired models (check the boxes).

2. Click "Train Selected Models".

3. Progress will be shown in the status bar.

4. Results will appear in the Results Tab automatically.

Understanding Results

After training, each model shows:

- RMSE: Root Mean Squared Error (lower is better).
- MAE: Mean Absolute Error (lower is better).
- R^2: Coefficient of determination (higher is better, max 1.0).
- Training Time: How long the training took.
- CV Score: Mean ± std cross-validation score.

> ⚡ **Pro Tip:**For materials science datasets, we recommend starting with Random Forest and XGBoost, as they often provide the best balance of accuracy and interpretability.

Neural Network Tab

The Neural Network Tab provides a visual interface for designing and training custom neural networks, now with full hyperparameter optimization support.

Architecture Design

Number of Hidden Layers

Use the spinbox to set 1-5 hidden layers. Each layer can be configured independently.

Layer Configuration

For each hidden layer, you can set:

- Units: Number of neurons (16-2048).
- Activation Function: ReLU, Leaky ReLU, Tanh, Sigmoid, GELU, ELU.
- Dropout Rate: Regularization (0.0-0.5).
- Batch Normalization: Enable/disable batch normalization.

Architecture Preview

Click "Update Architecture Preview" to see:

- Input layer dimensions.
- Hidden layer configurations.
- Output layer dimensions.
- Total number of parameters.
- Trainable parameters.

Training Configuration

Parameter	Description	Range
Epochs	Number of training iterations	10-10,000
Batch Size	Samples per gradient update	8-512
Learning Rate	Step size for optimization	0.00001-0.1
Optimizer	Optimization algorithm	Adam, AdamW, SGD, RMSprop
Weight Decay (L2)	Regularization strength	0.0-0.1
Early Stopping Patience	Stop if no improvement	5-500 epochs

Parameter	Description	Range
CV Folds	K-fold cross-validation folds	1-10 (default: 5)

Training Process

1. Configure architecture and training parameters.

2. Click "Train Neural Network".

3. Watch training progress:

- Progress bar shows current epoch.

- Log shows train/validation loss per epoch.

4. When CV Folds > 1, the trainer performs full K-fold cross-validation and reports mean ± std CV score.

5. Training stops automatically when:

- Maximum epochs reached, OR.
- Early stopping triggered (no improvement).

Neural Network Optimization

The Neural Network Tab now supports full hyperparameter optimization via Optuna, similar to scikit-learn regressors.

Optimization Settings

- Number of Trials: How many architecture combinations to try (10-200).
- CV Folds (Optimization): K-fold CV used during each trial (1-10, default: 5).

Custom Optimization Ranges

Click "Edit Optimization Ranges" to customize the search space for:

- Number of hidden layers.
- Units per layer.
- Dropout rate.
- Activation functions.
- Batch normalization toggle.
- Learning rate.
- Batch size.
- Weight decay.
- Optimizer selection.

Click "Reset Ranges to Default" to restore factory defaults.

Optimization Process

1. Configure optimization settings and ranges.

2. Click "Start Optimization".

3. The system explores architecture combinations via Bayesian optimization.

4. After completion, click "Apply Best Parameters to Architecture" to apply the optimal hyperparameters to the UI automatically.

5. Then click "Train Neural Network" to train with the optimized values.

Saving Neural Networks

After training:

1. Click "Save Model".

2. Choose format:

- PyTorch Model (.pt): Simple format.
- Model Bundle: Includes manifest.json, model, and preprocessor (recommended for Predict tab).

> ⚠ **Important:** Always save as Model Bundle if you plan to use the model in the Predict Tab. The bundle includes all necessary preprocessing information.

Optimization Tab

The Optimization Tab uses Bayesian optimization (Optuna) to find the best hyperparameters for your models.

Model Selection

Select a single model to optimize from the dropdown:

- Random Forest.
- Gradient Boosting.
- XGBoost.
- LightGBM.
- CatBoost.
- SVR.

Optimization Settings

- Number of Trials: How many hyperparameter combinations to try (10-500).
- Timeout: Maximum optimization time in seconds (0 = no limit).
- Enable Pruning: Stop unpromising trials early (recommended).
- CV Folds: K-fold cross-validation during optimization (1-10, default: 5).

Custom Parameter Ranges

Click "Edit Parameter Ranges" to customize the search space. For each parameter, you can set Min Value, Max Value, and Type (Integer or Float).

Available Parameters by Model

Model	Parameters
Random Forest	n_estimators, max_depth, min_samples_split, min_samples_leaf.
Gradient Boosting	n_estimators, max_depth, learning_rate, subsample.
XGBoost	n_estimators, max_depth, learning_rate, subsample, colsample_bytree, reg_alpha, reg_lambda.
LightGBM	n_estimators, max_depth, learning_rate, num_leaves, subsample, colsample_bytree, reg_alpha, reg_lambda.
CatBoost	iterations, depth, learning_rate, l2_leaf_reg.

Model	Parameters
SVR	C, epsilon, gamma.

Click "Reset to Default" to restore original ranges.

Optimization Process

1. Select model and configure settings.

2. (Optional) Edit parameter ranges.

3. Click "Start Optimization".

4. Monitor progress:

- Progress bar shows current trial.
- Log shows trial results in real-time.

Results

After optimization:

- Best Parameters: Optimal hyperparameter values found.
- Best Score: Best performance achieved.
- Number of Trials: Total trials completed.
- Optimization Time: Total time elapsed.
- Trial History Table: Complete history of all trials.

Train with Best Parameters

Click "Train with Best Parameters" to train a new model using the optimized hyperparameters from the best trial.

- The optimized model is registered automatically in the Results Tab.
- Appears as {ModelType}_Optimized with full test metrics and CV scores.

Saving Optimized Models

Click "Save Optimized Model":

- Choose format: Model Bundle (recommended) or Legacy Format.
- Select destination folder.
- Model is saved and appears in Results Tab.

> ⚡ **Pro Tip:** Start with 50 trials for initial exploration. Increase to 100-200 for production models.

Results Tab

The Results Tab displays all trained, optimized, and ensemble models across four dedicated sub-tabs.

Training Results

All trained models are shown in a table with:

- Model Name: Identifier.
- Type: Trained model.
- RMSE: Root Mean Squared Error.
- MAE: Mean Absolute Error.
- R^2: Coefficient of determination.
- CV Score: Mean ± std cross-validation score.

Optimization Results

Optimized models display:

- Model Name: {ModelType}_Optimized.
- Best Score: Score from the best Optuna trial.
- CV Score: Mean ± std of CV scores across best-trial folds.
- Number of Trials: Total optimization trials completed.
- Optimization Time: Total time elapsed.

Neural Network Results

Neural network models display:

- Model Name: Neural Network or NN_Optimized.
- Epochs: Training iterations completed.
- Final Loss: Training and validation loss.
- CV Score: Mean ± std cross-validation score (when CV folds > 1).

Ensemble Results

Ensemble models display:

- Ensemble Name: Auto-generated identifier.
- Ensemble Type: Weighted average or Stacking.
- Member Models: List of models included.
- Member Weights: Weight assigned to each member.
- Test Metrics: RMSE, MAE, R^2 on test set.
- CV Score: Mean ± std cross-validation score.

Actions

For each model, you can:

- Details: View detailed information about the model.
- Delete: Remove the model from the list.
- Export: Save as model bundle or legacy format.

Model Details

Clicking "Details" shows:

- Model type and source.
- Training/optimization time.
- All performance metrics.
- Best parameters (for optimized models).
- Feature importance (if available).
- Neural network architecture (for NN models).
- Member weights (for ensemble models).

Learning Curves

Select a model and click "Generate Learning Curves" to visualize training vs validation loss over epochs. This helps diagnose overfitting/underfitting.

Exporting Models

1. Select model from dropdown.

2. Click "Export Model Bundle".

3. Choose destination folder.

4. Bundle contains:

- manifest.json: Model metadata.
- pipeline.joblib or model.pt: Trained model.
- preprocessor.joblib: Preprocessing pipeline (for NN).

Ensemble Tab

The Ensemble Tab allows you to combine multiple models for improved predictions, now with weight optimization and stacking support.

Selecting Models

1. Click "Refresh Model List" to load available models.

2. Select multiple models from the list (Ctrl+Click for multiple).

3. Available sources:

- Trained models from Training Tab.
- Optimized models from Optimization Tab.
- Neural Network models from Neural Network Tab.

Ensemble Configuration

Ensemble Type

- Weighted Average: Equal or optimized weighted averaging of member predictions.
- Stacking: Ridge regression meta-learner trained on member predictions.

CV Folds

Set K-fold cross-validation folds for ensemble evaluation (1-10, default: 5). When > 1, the ensemble is evaluated via full K-fold CV.

Number of Models

Shows how many models are selected.

Creating Ensemble

1. Select 2 or more models.

2. Choose ensemble type.

3. Set CV folds.

4. Click "Create Ensemble".

5. Wait for evaluation to complete.

Weight Optimization

Click "Optimize Weights" to use Bayesian optimization (Optuna) to find optimal member weights:

- Configurable number of trials (10-200).

- CV Folds (Optimization): K-fold CV during weight search (1-10, default: 5).
- Reports optimized weight per member model.

After optimization, click "Apply Optimized Weights" to update the ensemble configuration.

Ensemble Results

After creation, you'll see:

- Ensemble Name: Auto-generated identifier.
- Ensemble Type: Weighted average or Stacking.
- Test Metrics: RMSE, MAE, R^2 on test set.
- Training Metrics: RMSE, MAE, R^2 on training set.
- CV Score: Mean ± std cross-validation score.

Saving Ensembles

Click "Save Ensemble Model":

- Choose format (Bundle or Legacy).
- Select destination folder.
- Ensemble appears in Results Tab and can be used in Explainability.

> ⚡ **Pro Tip:** Ensembles work best when combining models with different strengths. Try combining a tree-based model (Random Forest) with a gradient boosting model (XGBoost) and a neural network.

Explainability Tab

The Explainability Tab provides model interpretation tools including SHAP values, permutation importance, and partial dependence plots.

Model Selection

1. Click "Refresh Model List" to load available models.

2. Select a model from the dropdown.

3. Available models: Trained, Optimized, Neural Network, Ensemble.

> 💡 **Note:** For ensemble models, a graceful message will be shown as ensemble explainability is not supported. For NN optimized models, the system will guide you to train first.

Analysis Options

Select which analyses to run:

- Compute SHAP Values: Feature contributions using SHAP.
- Compute Permutation Importance: Feature importance via permutation.
- Compute Partial Dependence Plots: Feature effect visualization.
- Analyze Domain Applicability: Data coverage analysis.

Number of Features

Set how many top features to display (1-100, default: 10).

Running Analysis

1. Select model and options.

2. Click "Run Explainability Analysis".

3. Monitor progress in the log area.

4. Results appear in tabs below.

Results Tabs

Feature Importance

Table showing:

- Feature: Feature name.
- Model Importance: Built-in importance (if available).
- Permutation: Permutation importance score.

- SHAP: Mean absolute SHAP value.
- Average Rank: Combined ranking across methods.

> 💡 **Note:** For Neural Networks, Model Importance shows "N/A" as they don't have built-in feature importance.

SHAP Values

Summary of SHAP analysis:

- Global feature importance (top N features).
- Mean absolute SHAP value per feature.

> 💡 **Note:** SHAP values are computed using KernelExplainer for universal compatibility across all model types.

Partial Dependence

List of features analyzed with partial dependence plots.

> 💡 **Note:** Partial Dependence Plots are not available for Neural Networks due to sklearn limitations.

Domain Applicability

Analysis of model applicability domain:

- Training Samples: Number of training samples.
- Original Features: Number of input features.
- PCA Components: Number of PCA components used.
- Explained Variance: Variance captured by PCA.
- Thresholds: Leverage, Mahalanobis, k-NN distance thresholds.

Prediction Explanation

Explain individual predictions:

- Enter instance index (row number).
- Click "Explain Prediction".
- View: Predicted value, SHAP contributions (top features), Top positive/negative contributors.

> 🧪 **Materials Science Context:** Understanding which features drive predictions is crucial in materials science. For example, knowing that electronegativity differences strongly influence band gap predictions can guide material design.

Predict Tab

The Predict Tab allows you to make predictions on new data using saved models.

> ⚠ **Important:** The Predict Tab is a key differentiator of AutoAIM Studio. It enables standalone prediction mode for deploying trained models on new materials without retraining.

Loading a Model

1. Click "Load Model Bundle".

2. Select the folder containing:

- manifest.json (required).
- pipeline.joblib or model.pt (required).
- preprocessor.joblib (if applicable).

The model information will be displayed, including:

- Model type and algorithm.
- Feature names expected.
- Target column name.
- Model metrics (if available).

Loading Prediction Data

1. Click "Load Prediction Data".

2. Select your CSV or Excel file.

3. Data preview will appear.

Feature Validation

The system checks if all required features are present:

- Green: All features present.
- Yellow: Missing features can be auto-generated.
- Red: Missing features cannot be generated.

Auto-Generate Features

If your data has a formula column (e.g., "composition", "formula"):

- Check "Auto-generate composition features from formula column if missing."
- The system will generate 106 composition descriptors automatically.

Making Predictions

1. Ensure all required features are present (green status).

2. Click "Make Predictions".

Exporting Predictions

1. Click "Export Predictions".

- Choose save location.
- Predictions are saved as CSV with all original columns plus new prediction column.

> ⚡ **Pro Tip:** Use the Predict Tab to screen large materials databases. Train on a small, well-characterized dataset, then predict properties for thousands of candidate materials.

Troubleshooting

Common Issues

Cannot generate missing features automatically

Cause: Missing composition features and no formula column detected.

Solution:

- Ensure your prediction data has a column with chemical formulas.
- Check the "Auto-generate composition features" checkbox.
- The column name should contain "formula", "composition", or "structure".

Selected model does not have predict method

Cause: Model object is corrupted or not properly loaded.

Solution:

- Retrain the model.
- Check if model was saved correctly.
- For Neural Networks, ensure model was built before training.

Model 'X' not found in any source

Cause: Model was deleted or not properly stored.

Solution:

- Retrain the model.
- Save the model again after training.

SHAP values not available for this model

Cause: SHAP computation failed (usually for Neural Networks).

Solution:

- Ensure model has a working predict method.
- Try with a smaller dataset sample.
- Check console for detailed error messages.

Negative or incorrect CV scores in Results tab

Cause: Previous versions averaged CV scores across all trials, producing misleading values.

Solution: This is fixed in v1.1.0. The optimizer now stores CV fold scores from the best trial only.

Performance Tips

Speed Up Training

- Reduce CV folds: Use 3 instead of 5.
- Use GPU: Enable GPU acceleration for Neural Networks.
- Reduce trials: Use fewer optimization trials (20-30 instead of 50+).
- Sample data: Train on a subset for initial exploration.

Improve Accuracy

- Generate composition features: Use all 106 descriptors.
- Optimize hyperparameters: Run optimization with more trials.
- Create ensembles: Combine multiple models with weight optimization.
- Try different algorithms: Not all algorithms work equally well for all datasets.

Reduce Memory Usage

- Disable composition features: If not needed.
- Reduce batch size: For Neural Networks.
- Use fewer optimization trials: Each trial keeps results in memory.

Getting Help

If you encounter issues not covered here:

- Check the log output in each tab.
- Review the console/terminal for detailed error messages.
- Ensure your data is properly formatted (CSV/Excel).
- Verify all required columns are present.

Feature Reference

Composition Features (106 Descriptors)

The custom composition feature generator computes the following statistics for each chemical formula:

Atomic Properties

- Atomic number (mean, std, min, max, range, median, percentiles).
- Atomic mass (mean, std, min, max, range, median, percentiles).
- Covalent radius (mean, std, min, max, range, median, percentiles).
- Van der Waals radius (mean, std, min, max, range, median, percentiles).

Electronic Properties

- Electronegativity (mean, std, min, max, range, median, percentiles).
- Ionization energy (mean, std, min, max, range, median, percentiles).
- Electron affinity (mean, std, min, max, range, median, percentiles).

Physical Properties

- Density (mean, std, min, max, range, median, percentiles).
- Boiling point (mean, std, min, max, range, median, percentiles).
- Melting point (mean, std, min, max, range, median, percentiles).

Periodic Table Properties

- Group number (mean, std, min, max, range, median, percentiles).
- Period number (mean, std, min, max, range, median, percentiles).

Additional Features

- Number of unique elements.
- Total number of atoms.
- Element fractions.

These features are computed from elemental properties and provide a comprehensive representation of material composition for machine learning models.

> **Materials Science Context:** The 106 descriptors capture statistical distributions of elemental properties within a compound. For example, Fe2O3 would generate statistics based on the properties of Fe and O, weighted by their stoichiometric ratios.

Appendix: Model Algorithms

This appendix provides detailed information about each algorithm available in AutoAIM Studio.

Random Forest

- Type: Ensemble of decision trees.
- Strengths: Handles non-linear relationships, robust to outliers.
- Best for: General-purpose regression, feature importance.

Gradient Boosting

- Type: Sequential ensemble.
- Strengths: High accuracy, handles complex patterns.
- Best for: Competitive accuracy, medium-sized datasets.

XGBoost

- Type: Optimized gradient boosting.
- Strengths: Fast, regularized, handles missing values.
- Best for: Tabular data competitions, production systems.

LightGBM

- Type: Histogram-based gradient boosting.
- Strengths: Very fast, memory efficient.
- Best for: Large datasets, speed-critical applications.

CatBoost

- Type: Gradient boosting with ordered boosting.
- Strengths: Excellent categorical handling, reduced overfitting.
- Best for: Datasets with many categorical features.

SVR (Support Vector Regression)

- Type: Kernel-based method.
- Strengths: Effective in high dimensions, memory efficient.
- Best for: Small to medium datasets, non-linear relationships.

Ridge Regression

- Type: Linear regression with L2 regularization.
- Strengths: Simple, fast, prevents overfitting.

- Best for: Baseline model, linear relationships.

Neural Networks

- Type: Deep learning with customizable architecture.
- Strengths: Learns complex non-linear patterns.
- Best for: Large datasets, complex feature interactions.

Quick Reference Card

Keyboard Shortcuts

Action	Shortcut/Location
Load Data	Data Tab > Load Data button.
Prepare Data	Data Tab > Prepare Data button.
Train Models	Training Tab > Train Selected Models.
Save Model	Results Tab > Export Model Bundle.
Make Predictions	Predict Tab > Make Predictions.

Problem-Solution Guide

If you have this problem...	Do this...
Training is too slow.	Reduce CV folds or use GPU.
Poor model accuracy.	Generate composition features, try optimization.
Out of memory.	Reduce batch size, disable composition features.
Missing features in Predict.	Enable auto-generate composition features.
Model won’t load.	Check manifest.json exists in bundle folder.

Citation & License

How to Cite

If you use AutoAIM Studio in your research, please cite:

```
@software{autoaim_studio,   title = {AutoAIM Studio: Automated Machine Learning for Materials Science},      version = {1.1.0},      year = {2026},      url = {https://github.com/cesargabrielvera1-ux/AutoAIM-Studio.git},             doi = {10.5281/zenodo.14944043} }
```

The software is also available on Zenodo with DOI: 10.5281/zenodo.14944043.

License

AutoAIM Studio is released under the MIT License. You are free to use, modify, and distribute this software for academic and commercial purposes.

Acknowledgments

AutoAIM Studio builds upon the following open-source libraries:

- scikit-learn: Machine learning algorithms.
- XGBoost: Gradient boosting framework.
- LightGBM: Fast gradient boosting.
- CatBoost: Categorical boosting.
- PyTorch: Neural network framework.
- Optuna: Hyperparameter optimization.
- SHAP: Model interpretability.
- pymatgen: Materials analysis.

AutoAIM Studio

Automated Machine Learning for Materials Science

veradelagarza@quimica.unam.mx